

Special Topics Documentation

Prepared by: Rene Andre Bedonia Jocsing

Published: November 20, 2025

Deadline: November 28, 2025

This article provides the prescribed template for the special topics documentation. Please check the PDF version of this article for a more accurate template. For authors who know LaTeX, please see the source code of the template.

NAME OF YOUR PROJECT

AUTHOR ONE	AUTHOR TWO
STUDENT_NUMBER_ONE	STUDENT_USERNAME_TWO

Abstract

Provide a short summary describing the completed project: its purpose, core features, and final outcomes. For inspiration, look at the abstracts of journal articles in the field of computer science.

Table of Contents

- Introduction
 - Background and Motivation
 - Objectives and Success Criteria
 - Scope and Limitations
 - System Architecture and Methodology
 - Implementation Details
 - Distribution & Deployment
 - Code Repository
 - Usage
 - Testing and Quality Assurance
 - Results and Discussion
 - Recommendations
 - Licensing and Credits
 - Appendices
 - References
 - Acknowledgements
-

Introduction

Summarize the completed project. Include what was delivered versus what was proposed, any changes from the original proposal, and a high-level statement of success or failure.

Background and Motivation

Explain the context, motivating problems, and prior work or research that informed the project.

Objectives and Success Criteria

List the original objectives and the criteria used to judge whether the project met them. Indicate which objectives were fully met, partially met, or not met.

Scope and Limitations

Clearly state what the project includes and excludes. List technical, time, or resource constraints that affected the final deliverable.

System Architecture and Methodology

Describe the high-level architecture of the project. Include diagrams (or links to diagrams) and explain components, data flow, and interfaces.

- Components
- Data flow and communication
- Third-party services and libraries used

Implementation Details

Document important design decisions, data models, algorithms, and notable code modules. Include short code snippets or file/dir layout examples.

Distribution & Deployment

For other developers, provide exact instructions and artifacts for distributing and deploying the project. Include the following details:

- **Release artifacts:**
 - `release/project-v1.0.0.tar.gz`
 - `docker/your-image:tag`
- **Deployment options**
 - Local install (requirements, steps)
 - Container deployment (Docker Compose / Kubernetes manifests)
 - Cloud deployment (provider-specific steps)
- **Step-by-step deployment example**
 1. Pull or download release artifact
 2. Install prerequisites (runtime, environment variables)
 3. Run migrations or initial setup
 4. Start the service
- **Configuration & environment**

- Example `config.example.env`
- Required secrets and how to provision them
- **Rollback & upgrade procedure**
- **Distribution channels**
- **Checks & Validation**

Code Repository

List the canonical source repository and release locations. Provide branch and tag conventions, contribution instructions, and how to reproduce builds.

The canonical source repository should also contain a link to the live deployment of the project (if applicable).

For example, if the project has been deployed on GitHub, see this discussion:

<https://github.com/orgs/community/discussions/46986RL>

- **Canonical repository:** <https://github.com/ORG/REPO>
- **Main branches and conventions:**
 - `main` — production-ready
 - `develop` — integration
 - `feature/*` — feature branches
- **Releases & tags:** semantic versioning `vMAJOR.MINOR.PATCH`
- **How to clone and run locally:**

```
1 git clone https://github.com/ORG/REPO.git
2 cd REPO
3 git checkout v1.0.0
```

- **Reproducible build steps**
- **Contribution guidelines**

Other developers should be able to reproduce your project on their machine!

Usage

Provide commands and examples to quickly run and test the project locally or in staging. If project involves hardware, this is where you include your user manual.

```
1 # Example
2 pip install -r requirements.txt
3 cp .env.example .env
4 make run
```

Include simple example use-cases and expected behavior or output. If applicable, this is where you include recordings, pictures, and other evidences.

Testing and Quality Assurance

Document your testing strategy and results:

- Unit tests — how to run and coverage
- Integration tests
- End-to-end / system tests
- Manual QA checklist
- Test data / fixtures used
- Known bugs (open issues)

Results and Discussion

Report the final results: performance numbers, user testing findings, benchmarks, screenshots, logs, demo notes, etc. Compare outcomes to the success criteria.

Recommendations

Provide guidance for future developers, for example:

- Regular maintenance tasks (dependency updates, backups, hardware checks)
- Known technical debt and refactors
- Suggested enhancements and roadmap

Licensing and Credits

State the license used and list contributors, third-party libraries, and attributions.

Appendices

Attach or link release notes, full changelog, binary checksums, and additional documentation:

- Release notes — `docs/release-notes.md`
- Changelog — `CHANGELOG.md`
- Checksums — `release/SHA256SUMS`

References

List academic references, websites, and other resources used during the project (use APA 7th where relevant).

Acknowledgements

Thank people, mentors, or organizations that helped the project.